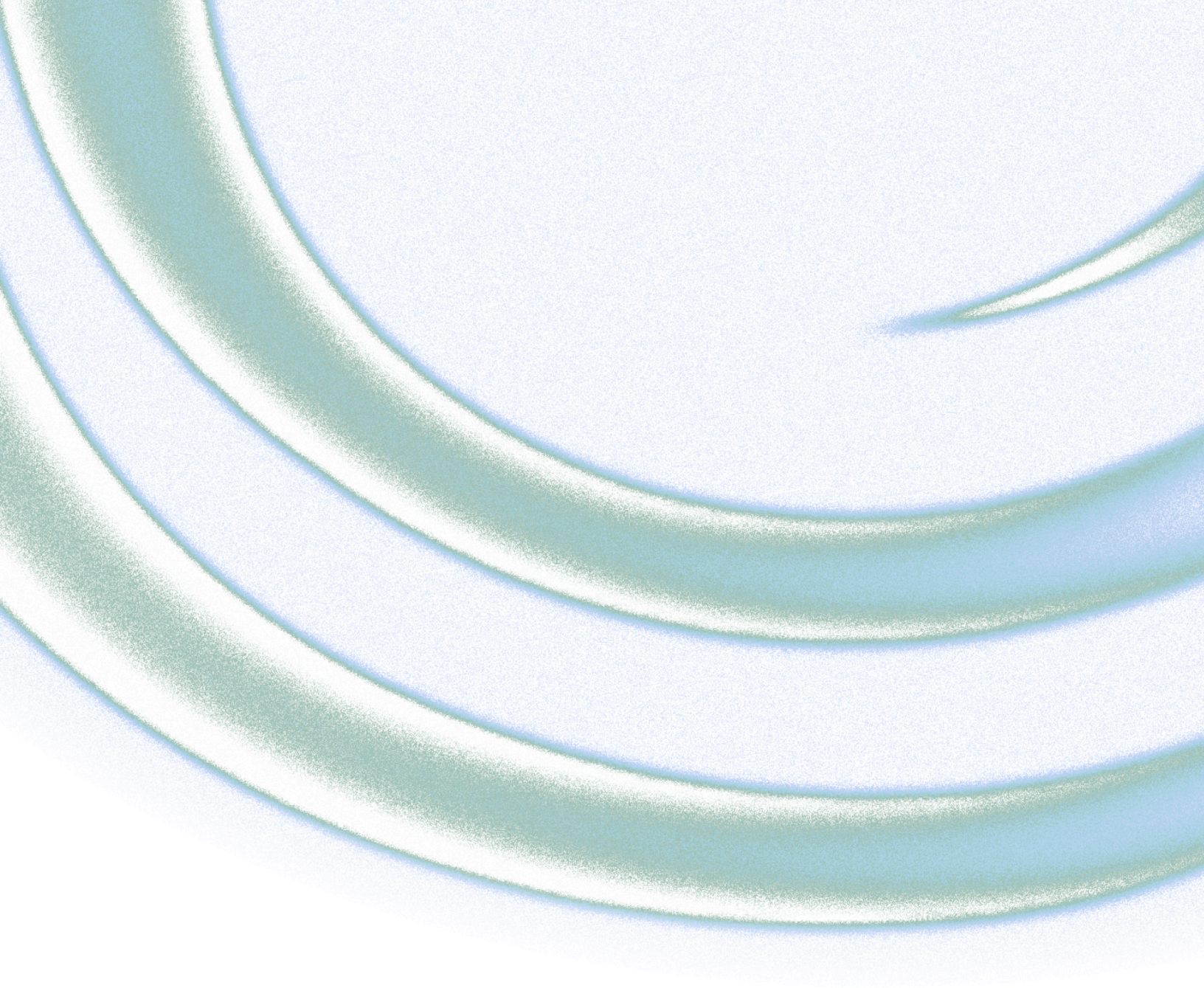


OtterSec

Solana Record Service

Security Assessment

Prepared for
Solana Foundation

Prepared by
Otter Audits, LLC

Prepared on
November 4th, 2026

Mingxuan Yang

ymxf00l@osec.io

Table of Contents

1. Executive Summary 2
    1.1. Overview 2
    1.2. Scope 2
A. Vulnerability Rating Scale 3
B. Procedure 4

© 2026 Otter Audits LLC. All Rights Reserved.

1. Executive Summary

1.1. Overview

Solana Foundation engaged OtterSec to conduct a security assessment of the `solana-record-service` program. This assessment was conducted between October 29th and October 30th, 2026. For information on our auditing methodology, refer to Appendix B.

No findings were produced throughout this audit engagement.

1.2. Scope

The Solana Record Service manages on-chain records under named classes, each governed by an authority. Records contain an owner, expiry timestamp, and data; owners can transfer or delete them, while class authorities manage their contents, expiry, and freezing. Records can optionally become Token-2022 tokens, allowing ownership to follow the token holder and revert to direct ownership when the token is burned. Permissioned classes require authority approval for record creation and grant the authority additional control over records.

The source code was delivered to us in a Git repository at `https://github.com/solana-foundation/solana-record-service`.

This audit was performed against commits `32a5654` and `ee7b214`.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the program's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.